
**< Large-Scale Design & Analysis of Neural
Networks >
System Requirements Specification
Version <2.0>
12/1/2024**

Document Control

Distribution List

The following list of people will receive a copy of this document every time a new version of this document becomes available:

Teaching assistants:

Clay Pate

Customer(s): M. Ilhan Akbas

John M. Thompson

Project team members:

Daniel Machado

Simon Cole

Muhammed Sharif Adeptu

Leah Powell

Change Summary

The following table details changes made between versions of this document:

Version	Date	Modifier	Description
0.1	10/15/2024	Leah	Document Creation
1.0	10/29/2024	Leah	Sections 1-7
1.1	11/14/2024	Leah	Sections 1.4, 7.2
1.2	11/20/2024	Muhammed-Sharif	Edits to sections 1.6, 2.2, 2.4
1.3	11/25/2024	Muhammed-Sharif	Sections. 2.5, 2.6, 2.7
1.4	11/26/2024	Leah	Sections 3-6
1.5	11/26	Simon	Added behavioral requirements
1.6	11/29	Daniel Machado	Section 3.1 User Interface Requirements
1.7	11/29	Daniel Machado	Added DFD level 0 1 and 2
2.0	12/1/2024	Leah Powell	3.3, 6.1, 6.2

Table of Contents

Document Control	ii
Distribution List.....	ii
Change Summary	ii
1. Introduction	1
1.1. Purpose and Scope	1
1.2. Intended Audience and Reading Suggestions	1
1.3. Document Conventions	1
1.4. Project References	1
1.5. Definitions, Acronyms, and Abbreviations	2
1.5.1. Definitions	2
1.5.2. Acronyms.....	2
1.5.3. Abbreviations	2
2. General Description	4
2.1. Product Perspective.....	4
2.2. Product Features	4
2.3. User Classes and Characteristics	4
2.3.1. Actors	5
2.3.2. Use Cases	5
2.3.3. Scenarios.....	6
2.4. General Constraints.....	6
2.5. Operating Environment.....	6
2.6. User Documentation	7
2.7. Assumptions and Dependencies	7
3. External Interface Requirements.....	8
3.1. User Interfaces	8
3.2. Software Interfaces.....	9
3.3. Communications Interfaces	9
4. Behavioral Requirements.....	10
4.1. Same Class of User.....	10
4.2. Stimulus.....	10
4.3. Related Features	10
4.4. Functional.....	11

5.	Non-behavioral Requirements.....	12
5.1.	Performance Requirements	12
5.2.	Safety Requirements	12
5.3.	Qualitative Requirements	12
5.3.1.	Availability.....	12
5.3.2.	Security.....	12
5.3.3.	Maintainability	12
5.3.4.	Portability.....	12
5.4.	Design and Implementation Constraints	13
6.	Other Requirements.....	14
6.1.	Database Requirements	14
6.2.	Operations.....	14
7.	Analysis Models.....	15
7.1.	<i>Data Flow Model</i>	15
7.1.1.	<i>Data Sources</i>	15
7.1.2.	<i>Data Sinks</i>	15
7.1.3.	<i>Data Dictionary</i>	15
7.1.4.	<i>Context Diagram (Level 0 Data Flow Diagram)</i>	16
7.1.5.	<i>Level 1 Data Flow Diagram</i>	16
7.1.6.	<i>Level 2 Data Flow Diagram</i>	17
7.2.	<i>Class Model</i>	18
7.3.	<i>State Model</i>	19
8.	To Be Determined List.....	20

1. Introduction

1.1. Purpose and Scope

This SRS covers the scope of the Large-Scale Design & Analysis of Neural Networks senior design project which aims to streamline the process of implementing deep learning using Bayesian Neural Networks. This is to address the lack of systemic optimization of a Neural Network's architecture from a statistical standpoint using the gradient of cost functions. This includes creating visualization tools for determining loss criteria and using histograms to analyze the weight distributions and convergence success for the neural networks created in this pipeline process.

The application of this system would be beneficial to the goals of any field in neural networks science. This is especially regarding the long-term accuracy of the training that results from these neural networks and the much-needed organization required to make these pipelined neural networks more efficient and consistent in accuracy/convergence.

1.2. Intended Audience and Reading Suggestions

This SRS document is intended to be used by any stakeholders of the project, members of the development team, or other collaborative parties such as teaching assistants. The rest of the SRS contains a product descriptive section (2.0) followed by a series of types of requirements. These include behavioral requirements, non-behavioral requirements, and any other non-specified requirements that might be necessary such as database requirements. After that are the project models which were specifically selected as appropriate for this project type and design. A typical reading sequence would be to start with the Introduction and Description, then read through the Analysis Models, then finally the requirements sections as directed.

1.3. Document Conventions

This document follows the typical template conventions.

1.4. Project References

S. Adepetu, S. Cole, D. Machado, and L. Powell, *Project Proposal Document*. 2024.

S. Adepetu, S. Cole, D. Machado, and L. Powell, *Software Design Documentation*. 2024.

1.5. Definitions, Acronyms, and Abbreviations

1.5.1. Definitions

This section lists terms used in this document and their associated definitions.

Table 1: Term Definitions

Term	Definition
Pytorch	A machine-learning library based on the open-source library on Lua.
Baye's Theorem	A theorem on probability, or statistical inference, that prior probability can be combined with new values and evidence to create a posterior probability which is updated as it becomes available.
Bayesian Neural Networks	Neural Networks whose weight distributions are modeled after probabilities using Baye's Theorem.
Architecture	Structural and logical layout and overall organization of a Neural Network.
Bins	Grouping continuous data into categories or intervals to use data as if it were discrete.
Weights	The connections between nodes/units in a neural network.
Histogram	Visualizing the distribution of continuous data into a graph and grouping into bins, similar to a bar-graph.
Convergence	Refers to the point where a model/network [learning] reaches a stable state and changes to its performance are at a minimum.
Tensors	Multidimensional arrays to store data used by neural networks

1.5.2. Acronyms

This section lists the acronyms used in this document and their associated definitions.

Table 2: Acronym Definitions

Term	Definition
SDD	System Design Document
NN	Neural Network
BNN	Bayesian Neural Network
BN	Batch Normalization
DL	Deep Learning
SRS	Software Requirements Specification
ML	Machine Learning
CONV	Convergence

1.5.3. Abbreviations

This section lists the abbreviations used in this document and their associated definitions.

Table 3: Abbreviations Definitions

Term	Definition
e.g.	For example
Etc.	Ex cetera

2. General Description

2.1. Product Perspective

This product serves to improve the structural organization and implementation of designing Neural Networks, or Deep Learning Systems. Results from this product will affect development in areas such as:

Network generation: Providing tools for automated creation of diverse network architectures based on user-defined parameters.

Networks initialization: Incorporating probabilistic methods to improve weight initialization and enhance training outcomes.

Pipeline optimization: Offering a structured framework to evaluate and improve network convergence and accuracy over time.

The core objective is to streamline the development process, making it easier for researchers and developers to build high-performing models efficiently.

2.2. Product Features

To achieve the goals outlined above, this product offers a suite of tools that empower users to analyze and optimize neural networks effectively. Key features include:

- **Network Loader:** A tool for quickly and efficiently loading a large set of neural networks into a project environment. This eliminates the manual effort of managing multiple architectures and datasets, enabling users to scale their experiments easily.
- **Loss Analysis Tool:** A visualization tool for analyzing loss curves to determine whether a network is converging effectively. This feature provides users with real-time feedback on their training results, helping them identify issues early in the process.
- **Weight Binning and Visualization:** A feature that bins network weights into discrete ranges and visualizes their distributions through histograms. This helps users understand the behavior of their network weights during training and assess whether the weights are stabilizing over time.
- **Success Criteria Evaluation:** A customizable tool that allows users to define success criteria for their networks, such as target accuracy, loss thresholds, or convergence speed. This ensures the generated networks meet user-defined performance benchmarks.
- **Export and Reporting:** Users can export training results, including loss graphs and weight distributions, in a standardized format for further analysis or sharing with collaborators.

2.3. User Classes and Characteristics

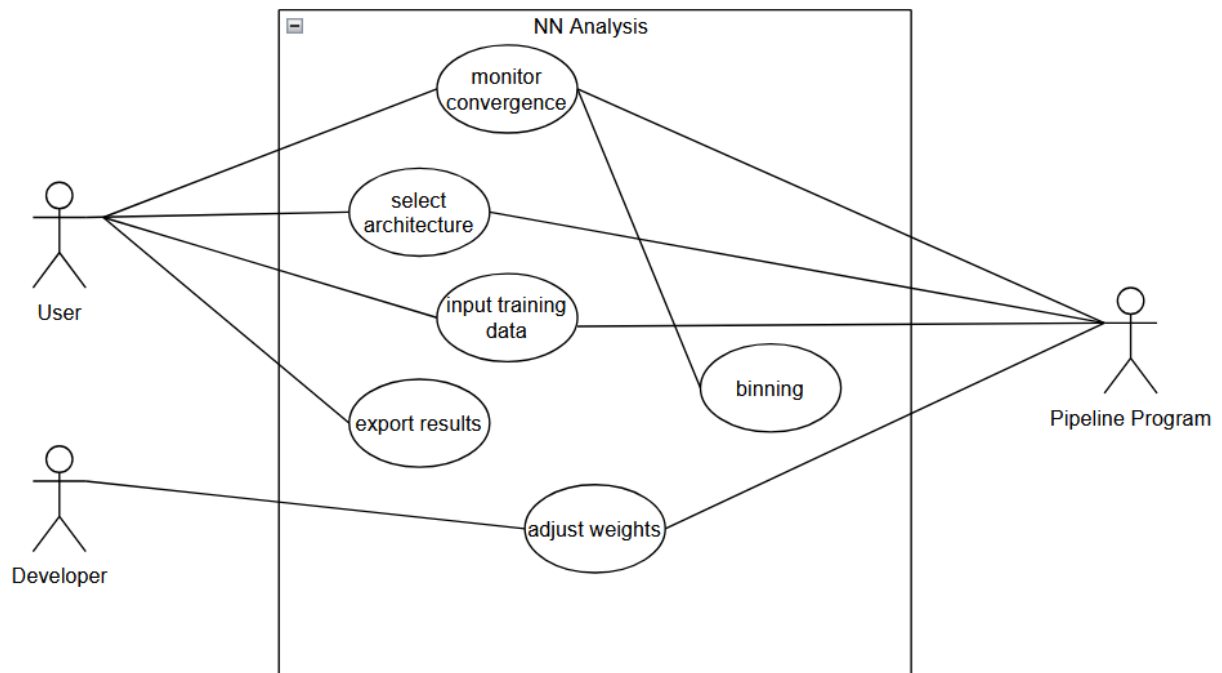


Figure 1. Use Case diagram of the system

2.3.1. Actors

This section presents the actors in the system.

- **User:**
 - An End User is the user which uses the system for enhancing their hiking experience.
- **Developer:**
 - A developer is the user which initially writes and later maintains the system
- **Pipeline Program:**
 - The program is an automated component of the system that manages the workflow of creating, training, and analyzing neural networks.

2.3.2. Use Cases

This section presents the Use Cases, developed for the system.

- **Monitor Convergence:** Track if networks meet defined success criteria.
- **Select Architecture:** Choose or define network structures.
- **Input Training Data:** Provide datasets for training.

- **Export Results:** Save analysis and training outcomes.
- **Adjust Weights:** Modify network weight parameters.
- **Binning:** Group and analyze network weights into ranges

2.3.3. Scenarios

*This section presents scenarios for **each use case**, described in the previous section.*

Scenario 1: Input Training Data

Description: *User inputs files to be used by the system to train the model*

Actors: *User, Pipeline Program*

Precondition: *The user has a .csv files prepared in program*

Trigger Condition: *User launches the program*

Steps:

1. *The system reads the config file to retrieve the file paths and parameters*
2. *The system ingests the input data from the .csv file in the data directory*
3. *The program parses the .csv file, extracting input (a, b_ and output (y) column.*
4. *The data is converted into PyTorch tensors for training.*
5. *The pipeline confirms ingestion process is complete & ready for the next stage.*

2.4. General Constraints

PyTorch Library Dependency:

- The system must rely on the PyTorch library, which requires compatibility with Python 3.8 or newer and installation of all necessary PyTorch dependencies.

- **Operating System Dependency:**

The system must be compatible with common operating systems like Windows 10, Ubuntu 20.04, or macOS Ventura to support a broad user base.

- **Resource Allocation:**

When training networks on the GPU, training tasks must optimize GPU resource usage to prevent bottlenecks, such as limiting batch sizes to avoid exceeding GPU memory.

2.5. Operating Environment

The software for the project that controls network creation runs in Jupyter Notebook and was tested using a macOS M2 chip to achieve the specific results presented. Similar results can be achieved on other systems with sufficient processing power and GPU capabilities.

2.6. User Documentation

User documentation components are currently not required or implemented.

2.7. Assumptions and Dependencies

Assumptions

1. Users have access to a compatible Python environment (Python 3.8 or newer) and can install the required libraries, including PyTorch.
2. Users are familiar with Jupyter Notebook as the primary interface for running the software.
3. The system assumes that users have basic knowledge of neural networks and their training processes.
4. The input training data provided by the user is clean, properly formatted, and free of significant errors or missing values.
5. Users have access to sufficient computational resources, such as a modern CPU or GPU, for running the training tasks efficiently.

Dependencies

1. The software is dependent on the PyTorch library for implementing and training neural networks.
2. CUDA is required for leveraging GPU acceleration during training on systems with NVIDIA GPUs.
3. The system depends on standard Python libraries, such as NumPy and Matplotlib, for data processing and visualization.
4. Internet access is required for installing software dependencies and accessing the GitHub repository hosting the source code.
5. The project relies on Jupyter Notebook as the development and execution environment for running the neural network pipeline.

3. External Interface Requirements

3.1. User Interfaces

The following are the requirements related to the initialization of the system

[REQ-UI1] The system terminal shall display to the user a prompt to input architecture.

[REQ-UI2] The system shall allow the user to input the convergence threshold.

[REQ-UI3] The system shall allow the user to specify the size of the test-set as a percentage of the training data, within a minimum of 10%.

[REQ-UI4] The system shall allow the user to customize the success loss to evaluate each model.

[REQ-UI5] The system shall allow the user to input the number of networks to produce.

[REQ-UI6] The system shall allow the user to input the max number of networks. If the user does not input the maximum number of networks, it shall default to 1.3 times the amount of networks to produce.

The following are requirements related to the training of the model.

[REQ-UI7] The system shall allow the user to input the test data loader.

[REQ-UI8] The system shall allow the user to input the training data loader.

[REQ-UI9] The system shall allow the user to input the number of epochs to train the networks for.

[REQ-UI10] The system shall allow the user to input the loss function to use.

[REQ-UI11] The system shall allow the user to input the learning rate to be used for training.

[REQ-UI12] The system shall allow the user to display the loss graph of a single trained network. This is to be done before starting the training of the many networks.

[REQ-UI13] The system shall allow the user to display the histogram for the weight distribution of a specific layer.

[REQ-UI14] The system shall allow the user to step through the weights.

[REQ-UI15] The system shall accept weight distributions as input in the form of a 4D array with dimensions: (layer, neuron, incoming weight, bin).

[REQ-UI16] The system shall allow the user to specify the traversal strategy using the parameter `walkthrough_strategy`.

[REQ-UI17] The system shall allow the user to select a specific layer for analysis using the parameter `layer`. The layer index shall be 0-indexed

[REQ-UI18] The system shall accept `bin_edges` as input, representing the edges of the bins for the selected layer to label the x-axis with actual values.

[REQ-UI19] The system shall allow the user to optionally specify the starting neuron for the walkthrough using the parameter `start_neuron`.

3.2. Hardware Interfaces

Not applicable to this project.

3.3. Software Interfaces

[REQ-SI1] The system shall run with files provided by the user for architecture and training data.

[REQ-SI2] The system shall interface with the operating system to validate and retrieve file paths for configuration files, data files, and model output directories.

[REQ-SI3] The system shall integrate with PyTorch for neural network training and inference.

[REQ-SI4] The system shall use pandas for reading and processing .csv files.

[REQ-SI5] The system shall output trained model state dictionaries using PyTorch.

3.4. Communications Interfaces

[REQ-CI1] The system shall be run using network connection.

4. Behavioral Requirements

4.1. Same Class of User

[REQ-SCU1] The system shall have only one level of access privilege to use across developers and users of the system.

4.2. Stimulus

[REQ-STIM1] The system shall automatically generate k Neural Networks based on architecture and training data provided by the user.

[REQ-STIM1.2] The system shall collect a number input from the user to generate a defined number of NNs.

[REQ-STIM1.3] The system shall train the generated networks with training data provided by the user.

[REQ-STIM1.4] The networks generated in requirement 1 shall be implemented in some user-defined NN architecture.

[REQ-STIM2] The system shall evaluate each neural network based on user-defined success criteria.

[REQ-STIM2.2] The system shall only save NNs, which meet a user defined loss-threshold.

[REQ-STIM2.3] The system shall save NNs which did not meet this threshold into a separate file.

[REQ-STIM2.4] The system shall display to the user the number of NNs successfully trained and the number failed.

[REQ-STIM3] The user shall be able to specify different training hyperparameters for each neural network generated, such as learning rate or batch size.

[REQ-STIM3.2] The system shall take in the NN learning rate as input.

[REQ-STIM3.3] The system shall take in the loss function to be used as input.

[REQ-STIM3.4] The system shall take in the loss threshold to define which networks succeed in training as input.

[REQ-STIM3.5] The system shall take in the maximum allowable difference in loss between the last two epochs as input.

4.3. Related Features

[REQ-RF1] The system shall have a tool which generates discretized bins for each weight of some NN.

X The system shall select each weight in the architecture of a set of NNs generated by the system.

The system shall find the minimum and maximum weights in each layer of the network architecture.

The system shall partition weights into equal width bins between the minimum weight value of the layer and the maximum.

The number of bins to partition into shall be provided by the user.

The system shall then iterate through every weight in all the networks and accumulate its corresponding bin.

[REQ-RF2] The system shall populate bins with the number of occurrences of that weight, in that range, across all NN's.

4.4. Functional

[REQ-F1] The system shall run without crashing.

[REQ-F2] The system shall alert the user if the maximum iterations is reached without success criteria being met.

[REQ-F3] The system shall allow the user to specify a maximum number of training epochs.

[REQ-F4] The system shall allow the user to specify a maximum number of attempts to generate a successful model.

[REQ-F5] The system shall allow the user to specify the loss criterion from the pytorch class.

[REQ-F6] The system shall be able to generate a graph displaying the loss of the neural network, calculated using the specified loss criterion.

[REQ-F7] The system shall be able to identify unique distributions.

[REQ-F8] the system should generate histograms according to the cumulation of weights.

5. Non-behavioral Requirements

5.1. Performance Requirements

[REQ-PR1] The system shall have a tool for systemic generation of loss graph.

[REQ-PR2] The system shall display the generated loss graph(s).

[REQ-PR3] The system shall use the loss graph tool to define the success criteria.

5.2. Safety Requirements

[REQ-SR1] The system shall not be used for exploitation in any way.

5.3. Qualitative Requirements

[REQ-QR1] The system should be capable of handling increased data volumes without significant degradation in performance.

[REQ-QR2] Results from neural network training and analysis should be consistent and reproducible. This involves using controlled random seeds where necessary and maintaining version control for network architecture variations.

5.3.1. Availability

[REQ-Av1] The user shall be able to re-run the model if the network fails.

5.3.2. Security

[REQ-S1] The system shall keep generated network data in a specified file.

5.3.3. Maintainability

[REQ-M1] The system shall be clean of unnecessary lines or code.

[REQ-M2] The system shall be commented significantly, allowing any potential future developer to quickly and efficiently.

5.3.4. Portability

[REQ-P1] The system shall be able to run after installation of package from git without extra installations.

5.4. Design and Implementation Constraints

[REQ-DIC1] The software shall be developed in Jupyter Notebook using Pytorch.

6. Other Requirements

6.1. Database Requirements

[REQ-DR1] The system shall store training data entities, including input features and target labels in a structured format that supports PyTorch tensor conversion.

[REQ-DR2] The system shall maintain data about each training session, including model configurations, hyperparameters, and loss metrics.

[REQ-DR3] The system shall support constraints on data integrity so that no null values or incorrect data types are allowed in the training data.

[REQ-DR4] The system shall define relationships between data entities such that each input tensor corresponds uniquely to its target tensor ensuring one-to-one mapping.

[REQ-DR5] The system shall allow frequent access to config files and training datasets to ensure they can be reused across multiple runs without negative effects on performance.

6.2. Operations

[REQ-O1] The user shall be able to save neural networks on a set location.

[REQ-O2] The user shall be able to keep networks that meet success criteria and disregard those that do not meet it.

[REQ-O1] The system shall support user-initiated operations by requiring the user to launch the program and provide valid config files.

[REQ-O2] The system shall implement and support data processing functions such as creating histograms of weight distributions.

[REQ-O3] The system shall save trained model states in .pt files.

[REQ-O4] The system shall allow unattended operations while automated training is occurring and the saving of models without requiring user interaction during runtime.

7. Analysis Models

7.1. Data Flow Model

<to be inserted eventually>

7.1.1. Data Sources

The data sources and their inputs to the system identified in the data flow model are as follows:

- Network Layers
- Binning mechanism
- Histogram
- Pipeline

7.1.2. Data Sinks

The data sinks and their system outputs identified in the data flow model are as follows:

- Network Layers
- Tools
- Network Program

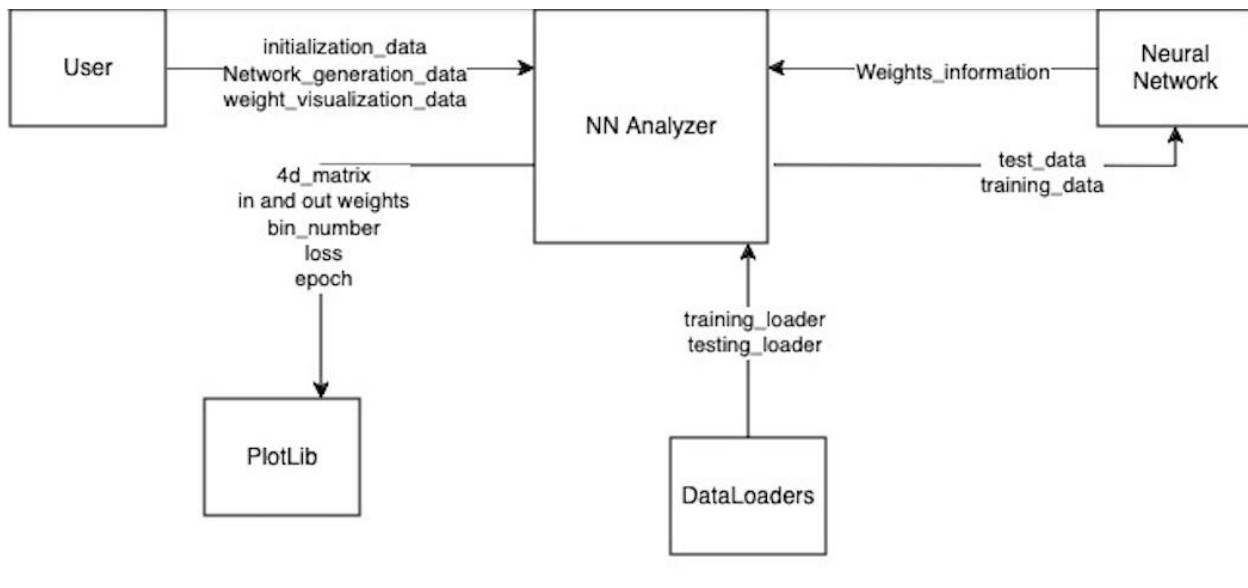
7.1.3. Data Dictionary

The data types are described in the data dictionary below. This section includes the name of the data type; a description of the contained data; how the data is structured; and the range of values.

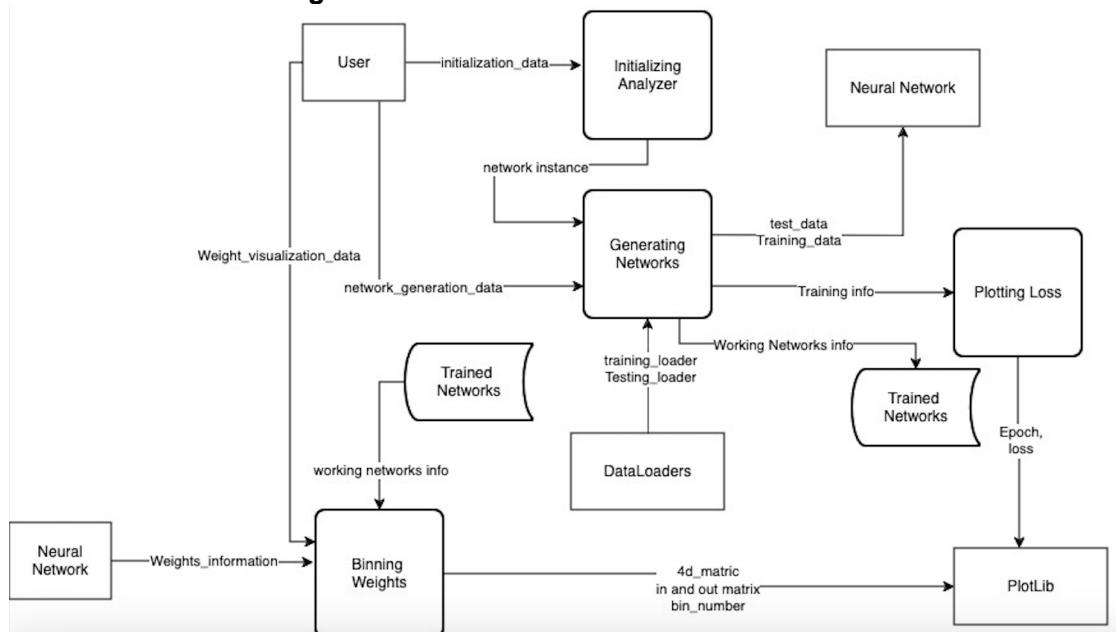
Name	Description	Structure	Range
Training data			
Number of networks			
Bins			
Epoch data/results			

<i>Initialization_data</i>	<i>Contains: neural_network_class, amount_to_produce, success_loss, convergence_threshold, max_attempts.</i>		
<i>Network_generation_data</i>	<i>Contains: num_epochs, loss_fn, learning_rate</i>		

7.1.4. Context Diagram (Level 0 Data Flow Diagram)

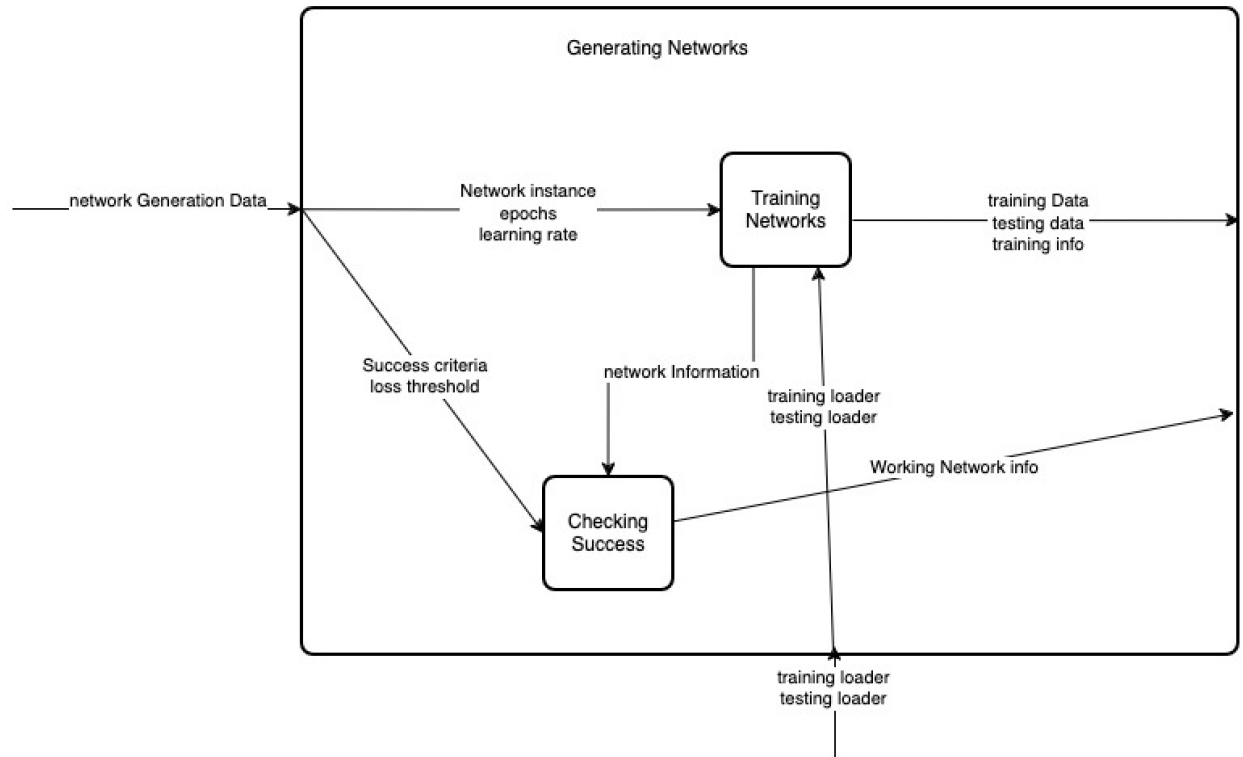


7.1.5. Level 1 Data Flow Diagram



7.1.6. Level 2 Data Flow Diagram

This is the level 2 DFD for the Generating Networks process



7.2. Class Model

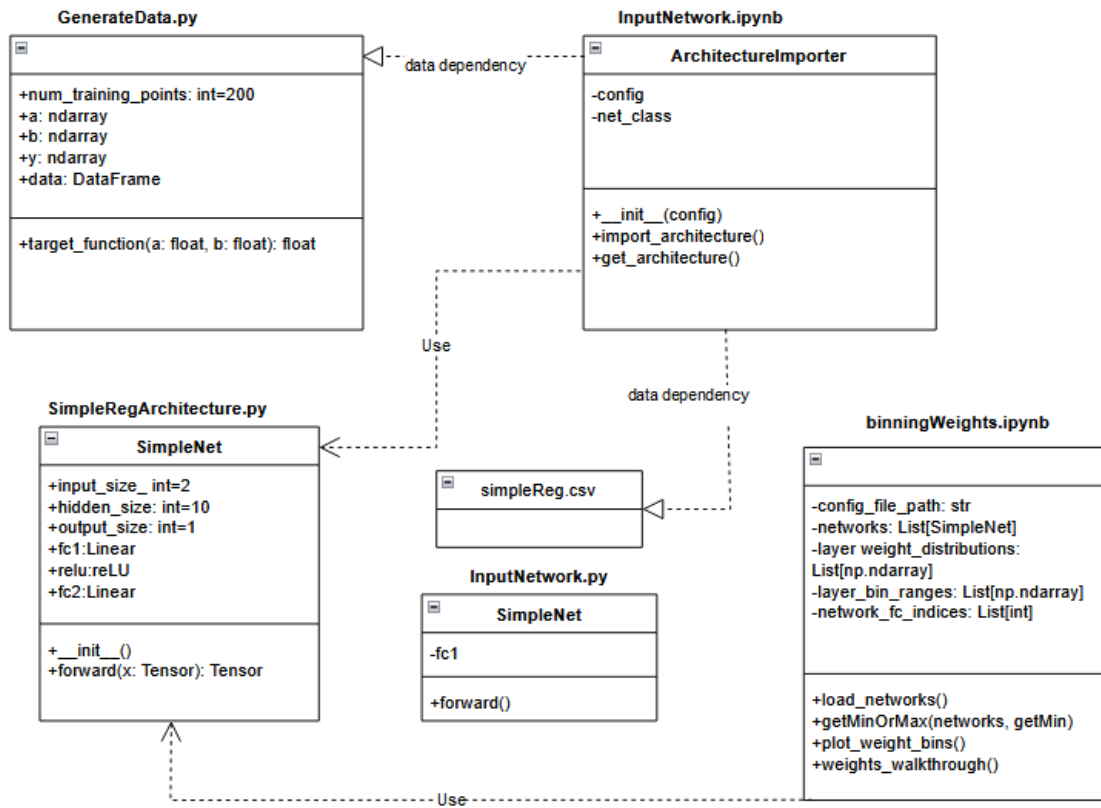


Figure: UML Diagram

7.3. State Model

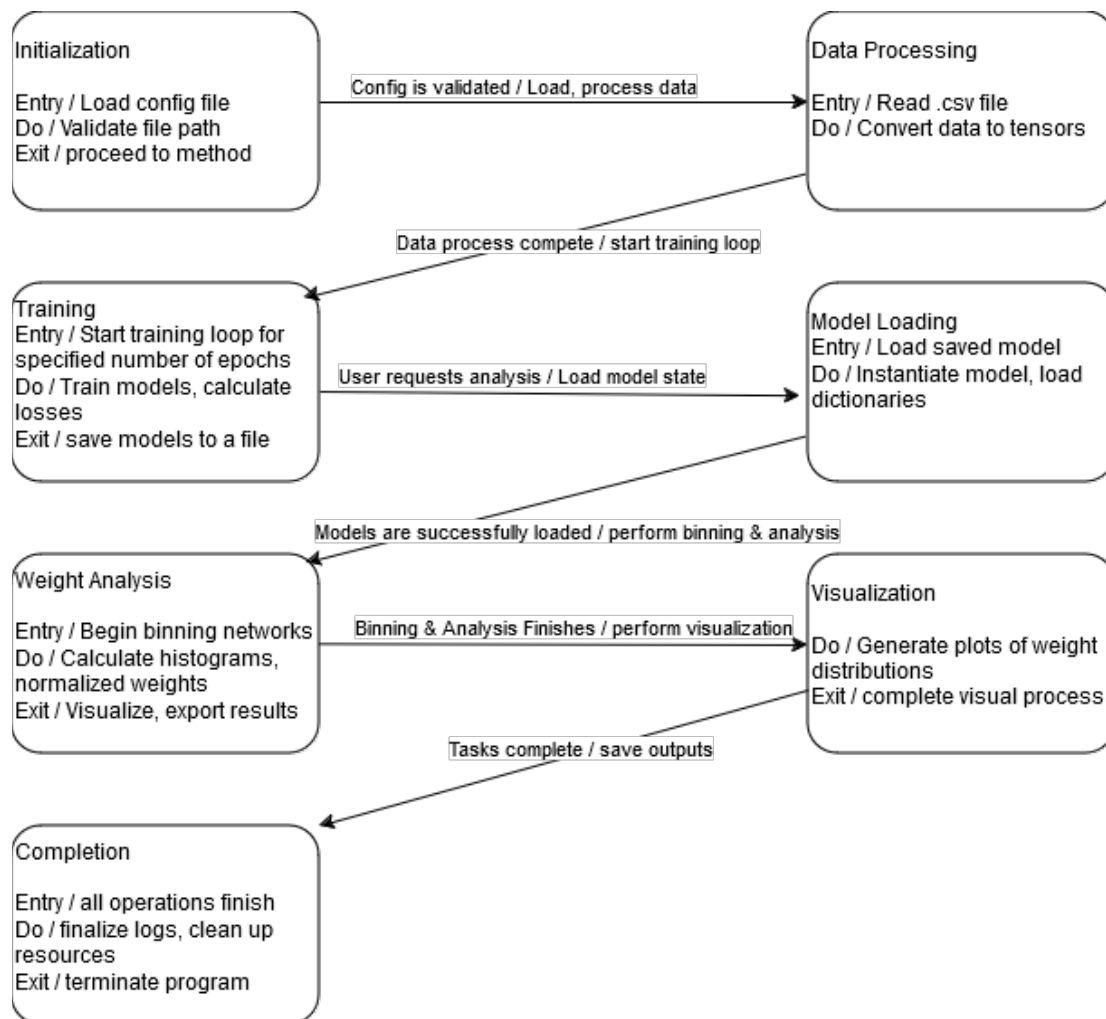


Figure: State Model Diagram

8. To Be Determined List